AWS vulnerable demo setup

Status: the provisioning, scanning, evidence, and cleanup code is prepared. Live acceptance is pending a dedicated AWS sandbox account and a saved worker credential. Do not enable this on an account containing production resources.

What the demo creates

Resource	Intentional misconfiguration	Exposure boundary
Empty S3 bucket	Bucket-level public access blocking disabled; anonymous-read bucket policy attempted	No objects or customer data uploaded; account-level blocking is never changed
IAM user	No MFA and a wildcard administrative inline policy	No access key or console password created
Dedicated VPC	No flow logs	No subnets, internet gateway, compute, or NAT gateway created
Security group in that VPC	Public SSH (22), RDP (3389), PostgreSQL (5432), and all-protocol ingress	Not attached to an instance or network interface

The worker persists resource IDs after each successful creation, with account ID, lab ID, and expiration tags. The existing scan worker scans the sandbox account, stores real findings and evidence, and the sandbox worker deletes the tracked resources after scan completion or the five-minute TTL. The dashboard opens the scan linked to this lab, not historical AWS results.

Account setup checklist

1. Create a separate AWS account under your organisation, or a standalone test account. Keep production/customer data out. Account creation and billing setup are administrator tasks, not part of the demo button.
2. Protect the account root user with MFA. Do not create root access keys.
3. Configure a small budget alert and select one region, such as `ap-south-1`. Budget alerts notify; they do not guarantee a spending cap.
4. Create a worker IAM identity named `CloudGuardSandboxWorker`, not `cg-*`. Prefer a dedicated assumable role; if using an IAM user, enter its credentials directly in CloudGuard's Manage Credentials, not in email or Git.
5. Grant scanner read permissions, such as AWS's `ReadOnlyAccess` managed policy in this **empty sandbox only**. This is broad account-level read access, not a least-privilege scanner policy.
6. Copy `worker-policy.template.json`, substitute `SANDBOX_ACCOUNT_ID` with the actual 12-digit account ID and `SANDBOX_REGION` with the selected region, and attach

the resulting additional write policy to the worker identity. It limits IAM writes to generated lab-user names, S3 writes to `cg-*` buckets, and EC2 writes to tagged lab resources. Have the administrator validate the policy in IAM Access Analyzer and check organisation SCPs/permission boundaries.

7. Save an AWS credential named `CloudGuard AWS Sandbox`, with the chosen region, and make it the default AWS credential for your CloudGuard account. A saved role ARN must reference the sandbox **write-capable** worker role, not the existing scanner read-only role. The source identity needs `sts:AssumeRole`, and role trust must permit it with the application's current external ID `CloudGuard-Scanner-Validation`.
8. In Dokploy, add `SANDBOX_AWS_ACCOUNT_IDS=<actual-account-id>` and leave `SANDBOX_AWS_DEPLOY=false` until credentials and isolation are checked. Compose passes both settings to the API and sandbox worker. No additional VM is needed for these AWS resources.

Enabling and accepting the first live run

After the sandbox owner approves the temporary public-policy/security-group misconfigurations, enable `SANDBOX_AWS_DEPLOY=true` and redeploy the scanner stack. The worker checks STS account identity against the allowlist **before creating resources** and refuses root credentials. Allowlisting alone does not prove the account is empty; the administrator must confirm its isolation.

Click AWS Demo and verify:

- The lab's proof account ID is the approved account and all four resource IDs are recorded before the scanner job starts.
- The scanner reports actual S3 access-block/versioning configuration, IAM MFA and wildcard-policy issues, open security-group rules, and VPC flow-log status.
- If account-level S3 public access blocking rejects the public policy, the proof records `public_bucket_policy_applied=false`. Do not disable account-level protection just to force this finding; bucket-level BPA findings still test it.
- Evidence is readable from object storage and available through the tenant's GRC connector. Expected lab findings are not a substitute for actual results.
- Only this demo's AWS scan IDs appear on the dashboard, with all findings.
- The lab reaches `destroyed`, with no cleanup errors, and AWS confirms the bucket, IAM user, security group, and VPC are gone. Previously deleted resources count as successful cleanup on retries.

If provisioning fails, the partial resource manifest remains in the database so the worker can retry cleanup. If the worker/VM is offline, TTL cleanup cannot run; an administrator must remove the tagged resources manually. Tags do not delete resources themselves. Cleanup refuses credentials targeting a different account. The code does not empty unexpectedly non-empty buckets or delete unrelated data.

Automated checks and current limitation

`python -m unittest backend.tests.test_aws_sandbox` exercises provisioning, account/root guards, role forwarding, partial failures, idempotent cleanup, permission errors, and public-bucket-policy detection using mocked AWS clients. No test in this suite creates real AWS resources. The permission template has not yet been tested against a live sandbox and may need adjustment for your SCPs.

AWS references

- GetBucketPolicyStatus
- S3 action permissions
- EC2 action/resource/tag permissions